Matrix Operations for Deep Learning

Neural networks and transformers, with every calculation shown

Akshay Redekar

October 18, 2026

Contents

1 Shapes and compatibility **3**

1.1 Which products are allowed? . . . 3

1.2 Shapes and parameter count of a small network . . . 3

1.3 Shapes inside a transformer attention head . . . 4

2 Matrix addition and scalar multiplication **5**

2.1 Residual connection in a transformer . . . 5

2.2 One step of gradient descent . . . 6

2.3 Scaling attention scores . . . 6

3 Matrix multiplication **7**

3.1 A neural-network layer on a batch . . . 7

3.2 Token lookup: picking a row of the embedding table . . . 8

3.3 The same layer as a mixture of rows . . . 9

3.4 Low-rank weight update (LoRA) . . . 9

3.5 How much work is a matrix product? . . . 10

4 Properties of matrix multiplication **11**

4.1 Order matters: XW versus WX . . . 11

4.2 Stacked linear layers collapse into one . . . 12

4.3 Splitting a sum: merging a LoRA update . . . 14

4.4 Grouping changes the amount of work . . . 14

5 Transpose **15**

5.1 Backpropagation through a linear layer . . . 15

5.2 Checking $(AB)^T = B^T A^T$ . . . 16

5.3 Attention scores QK^T . . . 17

6 Elementwise (Hadamard) product **18**

6.1 Gating . . . 18
6.2 Dropout . . . 19
6.3 Causal mask in attention: the order of operations . . . 19
6.4 Same two matrices, two different products . . . 20

7 Broadcasting **21**
7.1 Adding a bias to a layer output . . . 21
7.2 Layer normalization of tokens . . . 21
7.3 The silent bug: $(N, 1)$ minus (N) . . . 22

8 Special matrices you will meet **23**
8.1 Identity matrix . . . 23
8.2 Diagonal matrix: scaling each feature . . . 23
8.3 Lower-triangular matrix: the causal mask pattern . . . 24
8.4 Symmetric matrix: $X^T X$ . . . 24

9 Putting it together **25**
9.1 Forward pass of a two-layer network . . . 26
9.2 Masked self-attention for three tokens . . . 28

10 Formula sheet **29**

11 Practice problems **29**

12 Answers **30**

13 Self-check **31**

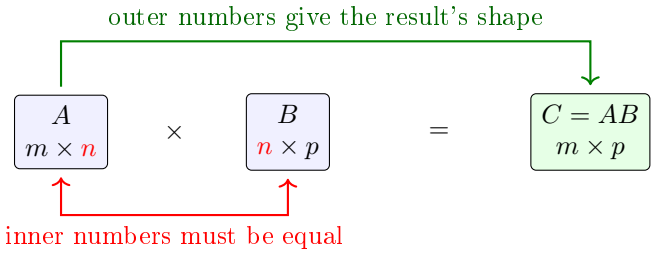
1 Shapes and compatibility

A matrix $A \in \mathbb{R}^{m \times n}$ has m rows and n columns. The entry in row i and column j is written a_{ij}.

In deep learning the convention is: **each row is one example, each column is one feature**. A batch of 5 examples with 4 features each is a 5×4 matrix.

Two matrices can be multiplied only when the **inner numbers match**. The result keeps the **outer numbers**:

$$(m \times n)\,(n \times p) \longrightarrow (m \times p).$$



1.1 Which products are allowed?

Calculation

Check the inner numbers each time.

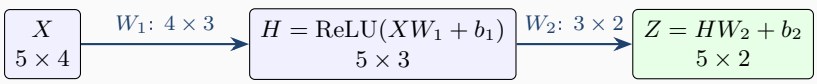
Product	Inner numbers	Allowed?	Result shape
$(2 \times 3)(3 \times 2)$	3 and 3	yes	2×2
$(3 \times 2)(2 \times 3)$	2 and 2	yes	3×3
$(4 \times 3)(3 \times 1)$	3 and 3	yes	4×1
$(2 \times 3)(2 \times 3)$	3 and 2	**no**	—
$(1 \times 4)(1 \times 4)$	4 and 1	**no**	—

The first two rows use the same two matrices in opposite order and give different shapes. Order matters (Section 4).

1.2 Shapes and parameter count of a small network

Calculation

A network takes 4 input features, has a hidden layer of 3 neurons, and an output layer of 2 neurons. A batch of 5 examples goes in.



Shape of each product:

$$XW_1 : (5 \times 4)(4 \times 3) = 5 \times 3$$

$$HW_2 : (5 \times 3)(3 \times 2) = 5 \times 2$$

Number of learnable values (parameters):

$$W_1 : 4 \times 3 = 12$$

$$b_1 : 3$$

$$W_2 : 3 \times 2 = 6$$

$$b_2 : 2$$

$$\text{Total} = 12 + 3 + 6 + 2 = 23$$

The batch size (5) never changes the parameter count. It only changes how many rows flow through.

1.3 Shapes inside a transformer attention head

Calculation

A sentence has $n = 4$ tokens. Each token is a vector of $d = 3$ numbers, so the input is $X \in \mathbb{R}^{4\times 3}$. One attention head turns each token into vectors of size $d_k = 2$ using three weight matrices W_Q, W_K, W_V, each 3×2.

$$Q = XW_Q : (4 \times 3)(3 \times 2) = 4 \times 2$$

$$K = XW_K : (4 \times 3)(3 \times 2) = 4 \times 2$$

$$V = XW_V : (4 \times 3)(3 \times 2) = 4 \times 2$$

$$K^T : 2 \times 4$$

$$QK^T : (4 \times 2)(2 \times 4) = 4 \times 4$$

$$(\text{weights})\, V : (4 \times 4)(4 \times 2) = 4 \times 2$$

The 4×4 matrix holds one number for every (token, token) pair. The final 4×2 output has one new vector per token.

Where this appears in deep learning

Every layer of a neural network is a matrix product, so most errors you meet are shape errors. Write the shape under each tensor before running code. One detail to remember: PyTorch's `nn.Linear` stores its weight as (outputs, inputs), so it computes XW^T rather than XW.

Key points to remember

- $A \in \mathbb{R}^{m\times n}$ means m rows and n columns. Data: rows are examples, columns are features.
- $(m \times n)(n \times p) = (m \times p)$. The inner numbers must match and then disappear.
- Parameters of a layer: (inputs $\times$ outputs) + outputs.
- The batch size is the number of rows. It never changes the weights.
- Even if AB is allowed, BA may be a different shape or not allowed at all.

Common mistakes

- Forgetting that `nn.Linear` stores weights as (outputs, inputs).
- Getting the shape right by luck with square matrices while the math is wrong.

2 Matrix addition and scalar multiplication

Two matrices are added entry by entry, and **they must have exactly the same shape**: $(A + B)_{ij} = a_{ij} + b_{ij}$.

Multiplying by a number (a scalar) multiplies every entry: $(\alpha A)_{ij} = \alpha\, a_{ij}$.

2.1 Residual connection in a transformer

Calculation

A transformer layer adds its input back to its output: $Y = X + F(X)$. Take 2 tokens with 3 numbers each:

$$X = \begin{bmatrix} 0.5 & -1.0 & 2.0 \\ 1.5 & 0.0 & -0.5 \end{bmatrix}, \qquad F(X) = \begin{bmatrix} 0.2 & 0.3 & -0.4 \\ -0.5 & 0.1 & 0.6 \end{bmatrix}.$$

Both are 2×3, so adding is allowed. Add matching positions:

$$y_{11} = 0.5 + 0.2 = 0.7$$

$$y_{12} = -1.0 + 0.3 = -0.7$$

$$y_{13} = 2.0 + (-0.4) = 1.6$$

$$y_{21} = 1.5 + (-0.5) = 1.0$$

$$y_{22} = 0.0 + 0.1 = 0.1$$

$$y_{23} = -0.5 + 0.6 = 0.1$$

$$Y = \begin{bmatrix} 0.7 & -0.7 & 1.6 \\ 1.0 & 0.1 & 0.1 \end{bmatrix}.$$

If $F(X)$ were 2×4, the addition would be impossible. That is why transformer layers keep the same width in and out.

2.2 One step of gradient descent

Calculation

Training changes a weight matrix by $W \leftarrow W - \eta G$, where G is the gradient (same shape as W) and $\eta = 0.1$ is the learning rate.

$$W = \begin{bmatrix} 0.50 & -0.20 \\ 0.10 & 0.80 \\ -0.30 & 0.40 \end{bmatrix}, \qquad G = \begin{bmatrix} 0.20 & -0.10 \\ 0.50 & 0.30 \\ -0.40 & 0.60 \end{bmatrix}.$$

Step 1: scalar multiplication $0.1\,G$.

$$0.1(0.20) = 0.02 \qquad 0.1(-0.10) = -0.01$$

$$0.1(0.50) = 0.05 \qquad 0.1(0.30) = 0.03$$

$$0.1(-0.40) = -0.04 \qquad 0.1(0.60) = 0.06$$

Step 2: subtract from W entry by entry.

$$0.50 - 0.02 = 0.48 \qquad -0.20 - (-0.01) = -0.19$$

$$0.10 - 0.05 = 0.05 \qquad 0.80 - 0.03 = 0.77$$

$$-0.30 - (-0.04) = -0.26 \qquad 0.40 - 0.06 = 0.34$$

$$W_{\text{new}} = \begin{bmatrix} 0.48 & -0.19 \\ 0.05 & 0.77 \\ -0.26 & 0.34 \end{bmatrix}.$$

2.3 Scaling attention scores

Calculation

Attention scores are divided by $\sqrt{d_k}$. For $d_k = 4$ this is $\sqrt{4} = 2$, so every entry is multiplied by $\frac{1}{2}$:

$$S = \begin{bmatrix} 4 & 2 \\ 6 & 8 \end{bmatrix} \Rightarrow \tfrac{1}{2}S: \quad \begin{matrix} \tfrac{1}{2}(4) = 2 & \tfrac{1}{2}(2) = 1 \\ \tfrac{1}{2}(6) = 3 & \tfrac{1}{2}(8) = 4 \end{matrix} \Rightarrow \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}.$$

Without this scaling, scores grow with d_k, and softmax would put almost all weight on one token.

Where this appears in deep learning

Weight updates, residual connections, and score scaling are all additions and scalar multiplications. Averaging model weights is the same operation with several matrices.

Key points to remember

- Addition needs identical shapes, and the result has that same shape.
- $A + B = B + A$ and $(A + B) + C = A + (B + C)$.
- $\alpha(A + B) = \alpha A + \alpha B$ and $(\alpha + \beta)A = \alpha A + \beta A$.
- The zero matrix changes nothing when added; $-A$ cancels A.
- Residual connection: output $= X + F(X)$, which forces $F(X)$ to have the shape of X.
- Gradient step: new weights $= W - \eta G$, with G the same shape as W.

Common mistake

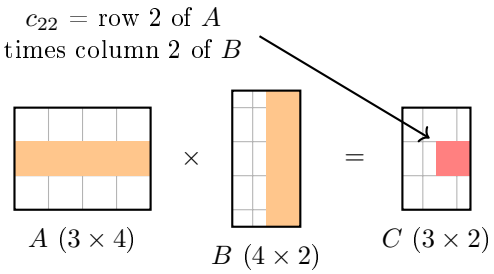
- Adding matrices of different shapes. Libraries may quietly repeat values to make the shapes fit (Section 7), which is not matrix addition.

3 Matrix multiplication

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$ the product $C = AB$ is $m \times p$ with

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj} = \sum_{k=1}^{n} a_{ik}b_{kj}.$$

In words: take row i of A and column j of B, multiply matching entries, and add them up.



3.1 A neural-network layer on a batch

Calculation

A layer has 3 inputs and 2 neurons. Two examples go in:

$$X = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 3 & 2 \end{bmatrix}, \quad W = \begin{bmatrix} 0.5 & -1 \\ 1 & 0.5 \\ -0.5 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 0.1 & -0.2 \end{bmatrix}.$$

Shapes: $(2 \times 3)(3 \times 2) = 2 \times 2$. Each entry of XW is a row of X times a column of W.

$$(XW)_{11} = 1(0.5) + 2(1) + (-1)(-0.5) = 0.5 + 2 + 0.5 = 3.0$$

$$(XW)_{12} = 1(-1) + 2(0.5) + (-1)(2) = -1 + 1 - 2 = -2.0$$

$$(XW)_{21} = 0(0.5) + 3(1) + 2(-0.5) = 0 + 3 - 1 = 2.0$$

$$(XW)_{22} = 0(-1) + 3(0.5) + 2(2) = 0 + 1.5 + 4 = 5.5$$

$$XW = \begin{bmatrix} 3.0 & -2.0 \\ 2.0 & 5.5 \end{bmatrix}.$$

Add the bias b to every row (Section 7 explains this repetition):

$$3.0 + 0.1 = 3.1 \qquad -2.0 + (-0.2) = -2.2$$

$$2.0 + 0.1 = 2.1 \qquad 5.5 + (-0.2) = 5.3$$

Apply ReLU, which replaces every negative number by 0:

$$\begin{bmatrix} 3.1 & -2.2 \\ 2.1 & 5.3 \end{bmatrix} \xrightarrow{\text{ReLU}} \begin{bmatrix} 3.1 & 0 \\ 2.1 & 5.3 \end{bmatrix}.$$

Neuron 2 is switched off for example 1.

3.2 Token lookup: picking a row of the embedding table

Calculation

A transformer stores one vector per word in an embedding table E (4 words, 3 numbers each):

$$E = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \end{bmatrix}.$$

A one-hot row $x = [0, 0, 1, 0]$ means "word 3". Shapes: $(1 \times 4)(4 \times 3) = 1 \times 3$.

$$(xE)_1 = 0(0.1) + 0(0.4) + 1(0.7) + 0(1.0) = 0.7$$

$$(xE)_2 = 0(0.2) + 0(0.5) + 1(0.8) + 0(1.1) = 0.8$$

$$(xE)_3 = 0(0.3) + 0(0.6) + 1(0.9) + 0(1.2) = 0.9$$

So $xE = [0.7, 0.8, 0.9]$, which is exactly row 3 of E. A vector with other numbers mixes rows. For $x = [0.5, 0.5, 0, 0]$:

$$(xE)_1 = 0.5(0.1) + 0.5(0.4) = 0.05 + 0.20 = 0.25$$

$$(xE)_2 = 0.5(0.2) + 0.5(0.5) = 0.10 + 0.25 = 0.35$$

$$(xE)_3 = 0.5(0.3) + 0.5(0.6) = 0.15 + 0.30 = 0.45$$

This is the average of rows 1 and 2. **Lesson:** a row vector times a matrix gives a mixture of the matrix's rows, with the vector's entries as the amounts.

3.3 The same layer as a mixture of rows

Calculation

Take example 1 from the first calculation, $x = [1, 2, -1]$, and the same W. Row k of W is scaled by x_k and the three scaled rows are added:

$$1 \cdot [0.5, -1] = [0.5, -1.0]$$

$$2 \cdot [1, 0.5] = [2.0, 1.0]$$

$$-1 \cdot [-0.5, 2] = [0.5, -2.0]$$

$$\text{sum} = [0.5 + 2.0 + 0.5, \;\; -1.0 + 1.0 - 2.0] = [3.0, -2.0]$$

This equals the first row of XW. A layer output is built from the layer's weight rows, so it can only reach combinations of those rows.

3.4 Low-rank weight update (LoRA)

Calculation

A column times a row is a matrix (an *outer product*). Let U be 3×2 and V be 2×4:

$$U = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 3 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 2 & 0 & 1 & 1 \end{bmatrix}.$$

The product UV (3×4) equals (column 1 of U)(row 1 of V) plus (column 2 of U)(row 2 of V).

Piece 1: $\begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 2 & 0 & 1 \end{bmatrix}$, each row is the matching number times $[1, 2, 0, 1]$:

$$\text{row } 1 = 1 \cdot [1, 2, 0, 1] = [1, 2, 0, 1]$$

$$\text{row } 2 = 2 \cdot [1, 2, 0, 1] = [2, 4, 0, 2]$$

$$\text{row } 3 = 0 \cdot [1, 2, 0, 1] = [0, 0, 0, 0]$$

Piece 2: $\begin{bmatrix}0\\1\\3\end{bmatrix}\begin{bmatrix}2 & 0 & 1 & 1\end{bmatrix}$:

$$\text{row } 1 = 0 \cdot [2,0,1,1] = [0,\ 0,\ 0,\ 0]$$

$$\text{row } 2 = 1 \cdot [2,0,1,1] = [2,\ 0,\ 1,\ 1]$$

$$\text{row } 3 = 3 \cdot [2,0,1,1] = [6,\ 0,\ 3,\ 3]$$

Add the two pieces:

$$UV = \begin{bmatrix}1 & 2 & 0 & 1\\2 & 4 & 0 & 2\\0 & 0 & 0 & 0\end{bmatrix} + \begin{bmatrix}0 & 0 & 0 & 0\\2 & 0 & 1 & 1\\6 & 0 & 3 & 3\end{bmatrix} = \begin{bmatrix}1 & 2 & 0 & 1\\4 & 4 & 1 & 3\\6 & 0 & 3 & 3\end{bmatrix}.$$

Why this matters. LoRA fine-tunes a 4096×4096 weight by learning the update as BA with B of size 4096×8 and A of size 8×4096. The update is a sum of only 8 outer products.

$$\text{full matrix} : 4096 \times 4096 = 16{,}777{,}216 \text{ values}$$

$$B \text{ and } A : 4096 \times 8 + 8 \times 4096 = 32{,}768 + 32{,}768 = 65{,}536 \text{ values}$$

$$\text{fraction} = \frac{65{,}536}{16{,}777{,}216} = 0.0039 \approx 0.39\%$$

3.5 How much work is a matrix product?

Calculation

An $(m \times k)(k \times n)$ product has $m \cdot n$ entries, and each needs k multiplications. So the work is $m \cdot k \cdot n$ multiplications.

$$XW \text{ above: } (2 \times 3)(3 \times 2) :\ 2 \cdot 3 \cdot 2 = 12$$

$$QK^T \text{ with } n = 4,\ d_k = 2 :\ (4 \times 2)(2 \times 4) :\ 4 \cdot 2 \cdot 4 = 32$$

$$QK^T \text{ with } n = 8,\ d_k = 2 :\ (8 \times 2)(2 \times 8) :\ 8 \cdot 2 \cdot 8 = 128$$

Doubling the number of tokens ($4 \to 8$) multiplies the attention-score work by $128/32 = 4$. This is why long inputs are expensive for transformers.

Where this appears in deep learning

Dense layers are $XW + b$. Embedding lookup is a one-hot row times the embedding table. Attention scores are QK^T. LoRA updates are products of two thin matrices.

Key points to remember

- c_{ij} = row i of A times column j of B, with the products added.
- Row vector times matrix = a mixture of the matrix's rows.

- Column times row = a full matrix. A sum of a few such pieces is a "low-rank" matrix.
- Work for $(m \times k)(k \times n)$ is mkn multiplications.
- Row vector times matrix is the special case with $m = 1$. Dot product is $m = n = 1$.

Common mistakes

- Multiplying matching entries only. That is the elementwise product (Section 6), not matrix multiplication.
- Writing UV when both factors store one item per row. You usually need UV^T.

4 Properties of matrix multiplication

For compatible shapes:

- **Grouping does not matter:** $(AB)C = A(BC)$.
- **Splitting a sum:** $A(B + C) = AB + AC$ and $(A + B)C = AC + BC$.
- **Identity:** $AI = IA = A$.
- **Order does matter:** in general $AB \neq BA$.

4.1 Order matters: XW versus WX

Calculation

Use $X = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 3 & 2 \end{bmatrix}$ (2×3) and $W = \begin{bmatrix} 0.5 & -1 \\ 1 & 0.5 \\ -0.5 & 2 \end{bmatrix}$ (3×2) from before. We found XW (2×2) already. Both orders are allowed, so compute WX: $(3 \times 2)(2 \times 3) = 3 \times 3$.

$$(WX)_{11} = 0.5(1) + (-1)(0) = 0.5$$

$$(WX)_{12} = 0.5(2) + (-1)(3) = 1 - 3 = -2$$

$$(WX)_{13} = 0.5(-1) + (-1)(2) = -0.5 - 2 = -2.5$$

$$(WX)_{21} = 1(1) + 0.5(0) = 1$$

$$(WX)_{22} = 1(2) + 0.5(3) = 2 + 1.5 = 3.5$$

$$(WX)_{23} = 1(-1) + 0.5(2) = -1 + 1 = 0$$

$$(WX)_{31} = -0.5(1) + 2(0) = -0.5$$

$$(WX)_{32} = -0.5(2) + 2(3) = -1 + 6 = 5$$

$$(WX)_{33} = -0.5(-1) + 2(2) = 0.5 + 4 = 4.5$$

$$XW = \begin{bmatrix} 3 & -2 \\ 2 & 5.5 \end{bmatrix} \ (2 \times 2), \qquad WX = \begin{bmatrix} 0.5 & -2 & -2.5 \\ 1 & 3.5 & 0 \\ -0.5 & 5 & 4.5 \end{bmatrix} \ (3 \times 3).$$

Different size, different numbers. The two orders mean different things: XW mixes features for every example, while WX would mix examples.

4.2 Stacked linear layers collapse into one

Calculation

Three layers with no activation function: W_1 is 3×4, W_2 is 2×3, W_3 is 1×2, acting on a column x:

$$W_1 = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & -1 \\ 2 & 1 & 0 & 0 \end{bmatrix}, \; W_2 = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \; W_3 = \begin{bmatrix} 3 & -1 \end{bmatrix}, \; x = \begin{bmatrix} 1 \\ 2 \\ 0 \\ 1 \end{bmatrix}.$$

Way 1: one layer at a time.

$$W_1 x = \begin{bmatrix} 1(1) + 0(2) + 2(0) + 1(1) \\ 0(1) + 1(2) + 1(0) + (-1)(1) \\ 2(1) + 1(2) + 0(0) + 0(1) \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 4 \end{bmatrix}$$

$$W_2(W_1 x) = \begin{bmatrix} 1(2) + 2(1) + 0(4) \\ 0(2) + 1(1) + 1(4) \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

$$W_3(W_2 W_1 x) = 3(4) + (-1)(5) = 12 - 5 = 7$$

Way 2: combine the weights first. $W_3 W_2$ is $(1 \times 2)(2 \times 3) = 1 \times 3$:

$$(W_3 W_2)_1 = 3(1) + (-1)(0) = 3$$

$$(W_3 W_2)_2 = 3(2) + (-1)(1) = 5$$

$$(W_3 W_2)_3 = 3(0) + (-1)(1) = -1$$

Then $(W_3 W_2) W_1$ is $(1 \times 3)(3 \times 4) = 1 \times 4$:

$$\text{col } 1 = 3(1) + 5(0) + (-1)(2) = 1$$

$$\text{col } 2 = 3(0) + 5(1) + (-1)(1) = 4$$

$$\text{col } 3 = 3(2) + 5(1) + (-1)(0) = 11$$

$$\text{col } 4 = 3(1) + 5(-1) + (-1)(0) = -2$$

$$(W_3 W_2 W_1)\, x = 1(1) + 4(2) + 11(0) + (-2)(1) = 1 + 8 + 0 - 2 = 7$$

Both ways give 7, as the grouping rule promises. Three layers acted like one 1×4 matrix. **Without an activation function between them, extra layers add nothing.**

4.3 Splitting a sum: merging a LoRA update

Calculation

Let $x = [1, 2, 3]$, base weights $W = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}$ and update $\Delta W = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$.

Compute separately, then add:

$$xW = [1(1) + 2(2) + 3(0),\ \ 1(0) + 2(1) + 3(1)] = [5, 5]$$

$$x\,\Delta W = [1(0) + 2(1) + 3(1),\ \ 1(1) + 2(0) + 3(1)] = [5, 4]$$

$$xW + x\,\Delta W = [10, 9]$$

Merge first, then multiply:

$$W + \Delta W = \begin{bmatrix} 1 & 1 \\ 3 & 1 \\ 1 & 2 \end{bmatrix}$$

$$x(W + \Delta W) = [1(1) + 2(3) + 3(1),\ \ 1(1) + 2(1) + 3(2)] = [10, 9]$$

Same answer. After training, the adapter can be merged into W and costs nothing extra at inference time.

4.4 Grouping changes the amount of work

Calculation

Compute xAB for one token x (1×4096) with A of size 4096×8 and B of size 8×4096. Both groupings give the same numbers.

Group as $x(AB)$:

$$AB:\ 4096 \cdot 8 \cdot 4096 = 134{,}217{,}728$$

$$x(AB):\ 1 \cdot 4096 \cdot 4096 = 16{,}777{,}216$$

$$\text{total} = 150{,}994{,}944 \text{ multiplications}$$

Group as $(xA)B$:

$$xA:\ 1 \cdot 4096 \cdot 8 = 32{,}768$$

$$(xA)B:\ 1 \cdot 8 \cdot 4096 = 32{,}768$$

$$\text{total} = 65{,}536 \text{ multiplications}$$

Ratio: $150{,}994{,}944 \div 65{,}536 = 2304$. Multiply the small vector first.

Where this appears in deep learning

Order matters for layer order and for QK^T versus K^TQ. The collapse of stacked linear layers is why every network puts an activation function (ReLU, GELU) between matrix products. Splitting a sum is why LoRA adapters can be merged.

Key points to remember

- $(AB)C = A(BC)$: regrouping never changes the answer but can change the work a lot.
- $AB \neq BA$ in general. Sometimes BA is not even allowed.
- With a column vector, transformations apply right to left: $W_3W_2W_1x$ applies W_1 first.
- $A(B + C) = AB + AC$.
- $AI = IA = A$ and $A0 = 0$.
- $AB = 0$ does not force $A = 0$ or $B = 0$, and $AB = AC$ does not force $B = C$.
- Stacked linear layers equal one linear layer.

Common mistakes

- Expanding $(A + B)^2$ as $A^2 + 2AB + B^2$. The correct result is $A^2 + AB + BA + B^2$.
- Reading the order left to right when vectors are columns.

5 Transpose

The transpose swaps rows and columns: $(A^T)_{ij} = a_{ji}$. If A is $m \times n$, then A^T is $n \times m$. Rules:

$$(A^T)^T = A, \qquad (A + B)^T = A^T + B^T, \qquad (\alpha A)^T = \alpha A^T, \qquad \boxed{(AB)^T = B^T A^T}$$

Notice that the order reverses in the last rule. A matrix is *symmetric* when $A^T = A$.

5.1 Backpropagation through a linear layer

Calculation

For a layer $Z = XW$, training receives dZ from the next layer (the effect of each output on the loss, same shape as Z). It must produce dW (to update W) and dX (to pass back). The rules are

$$dW = X^T\, dZ, \qquad dX = dZ\, W^T.$$

Shapes already force the transposes: dW must be 3×2 like W, and $X^T dZ$ is $(3 \times 2)(2 \times 2) = 3 \times 2$. Use $X = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 3 & 2 \end{bmatrix}$, $W = \begin{bmatrix} 0.5 & -1 \\ 1 & 0.5 \\ -0.5 & 2 \end{bmatrix}$, and $dZ = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}$.

Weight gradient. $X^T = \begin{bmatrix} 1 & 0 \\ 2 & 3 \\ -1 & 2 \end{bmatrix}$.

$$dW_{11} = 1(1) + 0(-1) = 1$$
$$dW_{12} = 1(0) + 0(2) = 0$$
$$dW_{21} = 2(1) + 3(-1) = -1$$
$$dW_{22} = 2(0) + 3(2) = 6$$
$$dW_{31} = -1(1) + 2(-1) = -3$$
$$dW_{32} = -1(0) + 2(2) = 4$$

$$dW = \begin{bmatrix} 1 & 0 \\ -1 & 6 \\ -3 & 4 \end{bmatrix} \quad (3 \times 2).$$

Input gradient. $W^T = \begin{bmatrix} 0.5 & 1 & -0.5 \\ -1 & 0.5 & 2 \end{bmatrix}$, and $dX = dZ\, W^T$ is $(2 \times 2)(2 \times 3) = 2 \times 3$.

$$dX_{11} = 1(0.5) + 0(-1) = 0.5$$
$$dX_{12} = 1(1) + 0(0.5) = 1$$
$$dX_{13} = 1(-0.5) + 0(2) = -0.5$$
$$dX_{21} = -1(0.5) + 2(-1) = -2.5$$
$$dX_{22} = -1(1) + 2(0.5) = 0$$
$$dX_{23} = -1(-0.5) + 2(2) = 0.5 + 4 = 4.5$$

$$dX = \begin{bmatrix} 0.5 & 1 & -0.5 \\ -2.5 & 0 & 4.5 \end{bmatrix} \quad (2 \times 3) \quad \text{(same shape as } X\text{)}.$$

5.2 Checking $(AB)^T = B^T A^T$

Calculation

From before, $XW = \begin{bmatrix} 3 & -2 \\ 2 & 5.5 \end{bmatrix}$, so $(XW)^T = \begin{bmatrix} 3 & 2 \\ -2 & 5.5 \end{bmatrix}$. Now compute $W^T X^T$, which is $(2 \times 3)(3 \times 2)$:

$$(W^T X^T)_{11} = 0.5(1) + 1(2) + (-0.5)(-1) = 0.5 + 2 + 0.5 = 3$$

$$(W^T X^T)_{12} = 0.5(0) + 1(3) + (-0.5)(2) = 0 + 3 - 1 = 2$$

$$(W^T X^T)_{21} = -1(1) + 0.5(2) + 2(-1) = -1 + 1 - 2 = -2$$

$$(W^T X^T)_{22} = -1(0) + 0.5(3) + 2(2) = 0 + 1.5 + 4 = 5.5$$

The result $\begin{bmatrix} 3 & 2 \\ -2 & 5.5 \end{bmatrix}$ equals $(XW)^T$. Computing $X^T W^T$ instead is not even allowed: $(3 \times 2)(3 \times 2)$ has inner numbers 2 and 3.

5.3 Attention scores QK^T

Calculation

Two tokens with $d_k = 2$: $Q = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $K = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$. Row i of Q is query i, row j of K is key j. To dot every query with every key, flip K so keys become columns: $K^T = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$.

$$(QK^T)_{11} = 1(1) + 0(2) = 1$$

$$(QK^T)_{12} = 1(0) + 0(1) = 0$$

$$(QK^T)_{21} = 1(1) + 1(2) = 3$$

$$(QK^T)_{22} = 1(0) + 1(1) = 1$$

$$QK^T = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}.$$

Entry (i, j) is query i dotted with key j: how strongly token i looks at token j.

Where this appears in deep learning

Every backward pass through a linear layer uses $X^T dZ$ and $dZ\, W^T$. Attention uses QK^T to compare all token pairs at once. Papers that write data as columns instead of rows flip every formula by a transpose.

Key points to remember

- $(AB)^T = B^T A^T$. For three factors, $(ABC)^T = C^T B^T A^T$.

- For $Z = XW$: $dW = X^T dZ$ and $dX = dZ\,W^T$. Matching shapes tells you where the transposes go.
- $X^T X$ and XX^T are always symmetric.
- $x^T x$ is a single number; xx^T is a matrix.
- Entry (i, j) of QK^T is the dot product of query i and key j.

Common mistake

- Writing $(AB)^T = A^T B^T$ (wrong order, often not even allowed).

6 Elementwise (Hadamard) product

The **elementwise product** $A \odot B$ multiplies matching entries: $(A \odot B)_{ij} = a_{ij} b_{ij}$. Both matrices must have the same shape and nothing is added up. In code this is `A * B`, while the matrix product is `A @ B`.

6.1 Gating

Calculation

Gated layers (LSTM, GRU, GLU) multiply a signal by a gate whose entries lie between 0 and 1. A gate of 0 blocks, a gate of 1 lets everything pass.

$$g = \begin{bmatrix} 0.9 & 0.1 & 0.5 \\ 0.2 & 0.8 & 1.0 \end{bmatrix}, \qquad c = \begin{bmatrix} 2 & -4 & 6 \\ 1 & 3 & -2 \end{bmatrix}.$$

$$(g \odot c)_{11} = 0.9(2) = 1.8$$
$$(g \odot c)_{12} = 0.1(-4) = -0.4$$
$$(g \odot c)_{13} = 0.5(6) = 3.0$$
$$(g \odot c)_{21} = 0.2(1) = 0.2$$
$$(g \odot c)_{22} = 0.8(3) = 2.4$$
$$(g \odot c)_{23} = 1.0(-2) = -2.0$$

$$g \odot c = \begin{bmatrix} 1.8 & -0.4 & 3.0 \\ 0.2 & 2.4 & -2.0 \end{bmatrix}.$$

6.2 Dropout

Calculation

During training, dropout randomly switches off activations. With drop probability $p = 0.5$, a random 0/1 mask M is multiplied in, and survivors are scaled by $\frac{1}{1-p} = 2$ so the average size stays the same.

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 0.5 & 3 \end{bmatrix}, \qquad M = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}.$$

Step 1: $X \odot M$ (keep where the mask is 1).

$$\text{row 1}: \ 1(1),\ 2(0),\ 3(1),\ 4(1) = [1,\ 0,\ 3,\ 4]$$

$$\text{row 2}: \ 2(0),\ 1(1),\ 0.5(1),\ 3(0) = [0,\ 1,\ 0.5,\ 0]$$

Step 2: multiply by 2.

$$\text{row 1} = [2,\ 0,\ 6,\ 8]$$

$$\text{row 2} = [0,\ 2,\ 1,\ 0]$$

6.3 Causal mask in attention: the order of operations

Calculation

In a text-generating transformer, token i may only look at tokens $j \le i$. Take 3 tokens with scores

$$S = \begin{bmatrix} 2 & 1 & 0.5 \\ 1 & 3 & 0.2 \\ 0.5 & 1 & 2 \end{bmatrix}.$$

Correct way: replace the forbidden entries by $-\infty$, then apply softmax to each row (softmax turns scores into positive numbers that add to 1; $e^{-\infty} = 0$).

$$\text{row 1 : only token 1 allowed} \Rightarrow [1,\ 0,\ 0]$$

$$\text{row 2 : scores } (1,\ 3): \ e^1 = 2.718,\ e^3 = 20.086,\ \text{sum} = 22.804$$

$$\text{weights} = \left(\tfrac{2.718}{22.804},\ \tfrac{20.086}{22.804}\right) = (0.119,\ 0.881)$$

$$\text{row 3 : scores } (0.5,\ 1,\ 2): \ e^{0.5} = 1.649,\ e^1 = 2.718,\ e^2 = 7.389,\ \text{sum} = 11.756$$

$$\text{weights} = (0.140,\ 0.231,\ 0.629)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0.119 & 0.881 & 0 \\ 0.140 & 0.231 & 0.629 \end{bmatrix}.$$

Wrong way: multiply S by a 0/1 mask. Row 1 becomes (2, 0, 0), and softmax gives

$$e^2 = 7.389,\ e^0 = 1,\ e^0 = 1,\ \text{sum} = 9.389$$

$$\text{weights} = (0.787,\ 0.107,\ 0.107)$$

Token 1 now puts about 21% of its attention on future tokens, because $e^0 = 1$, not 0.

6.4 Same two matrices, two different products

Calculation

Let $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$.

$$(AB)_{11} = 1(5) + 2(7) = 19 \qquad (A \odot B)_{11} = 1(5) = 5$$

$$(AB)_{12} = 1(6) + 2(8) = 22 \qquad (A \odot B)_{12} = 2(6) = 12$$

$$(AB)_{21} = 3(5) + 4(7) = 43 \qquad (A \odot B)_{21} = 3(7) = 21$$

$$(AB)_{22} = 3(6) + 4(8) = 50 \qquad (A \odot B)_{22} = 4(8) = 32$$

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}, \qquad A \odot B = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}.$$

Where this appears in deep learning

Gates, dropout masks, and attention masks use the elementwise product. Activation functions (ReLU, sigmoid) also act on one entry at a time. Linear layers and attention scores use the matrix product. Mixing the two gives a result with the right shape and the wrong meaning.

Key points to remember

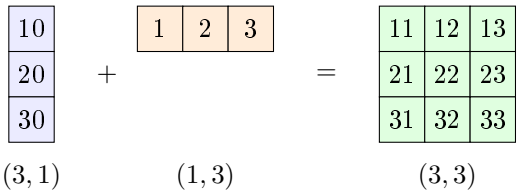
- $A \odot B$: same shape in and out, no adding up. `A * B`.
- AB: inner numbers must match and the products are summed. `A @ B`.
- The elementwise product is order-free ($A \odot B = B \odot A$); the matrix product is not.
- Dropout scales survivors by $\frac{1}{1-p}$ to keep the average the same.
- Attention masks: put $-\infty$ in forbidden positions *before* softmax. Never multiply scores by 0.
- A row that is fully masked gives `NaN` after softmax. Always leave at least one position open.

Common mistakes

- Typing `*` when `@` is meant. Library repetition rules hide the error.
- Masking after softmax without making each row add up to 1 again.

7 Broadcasting

Broadcasting lets libraries combine tensors of different shapes by repeating a size-1 (or missing) dimension. **Rule:** compare shapes from the last dimension backwards. Two sizes are compatible if they are equal or one of them is 1. A size of 1 is repeated to match.



The column is copied across 3 columns, the row is copied down 3 rows, and then the two 3×3 grids are added.

7.1 Adding a bias to a layer output

Calculation

The layer output XW has shape 2×2 and the bias b has shape (2). Compare from the right: 2 and 2 match; b has no first dimension, so it is copied for each row.

$$\begin{bmatrix} 3.0 & -2.0 \\ 2.0 & 5.5 \end{bmatrix} + \begin{bmatrix} 0.1 & -0.2 \\ 0.1 & -0.2 \end{bmatrix} = \begin{bmatrix} 3.1 & -2.2 \\ 2.1 & 5.3 \end{bmatrix}.$$

7.2 Layer normalization of tokens

Calculation

Transformers normalize each token vector: subtract its mean, then divide by its standard deviation. Take 2 tokens:

$$X = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 1 & 4 \end{bmatrix} \quad (2 \times 3).$$

The mean of each row has shape $(2, 1)$ and is copied across the 3 columns.

$$\text{mean}_1 = \frac{2+4+6}{3} = 4 \qquad \text{mean}_2 = \frac{1+1+4}{3} = 2$$

Subtract (row 1 minus 4, row 2 minus 2):

$$\text{row } 1 = [2-4,\ 4-4,\ 6-4] = [-2,\ 0,\ 2]$$

$$\text{row } 2 = [1-2,\ 1-2,\ 4-2] = [-1,\ -1,\ 2]$$

Standard deviation of each row (square, average, square root):

$$\text{row } 1: \ \frac{(-2)^2 + 0^2 + 2^2}{3} = \frac{8}{3} = 2.667, \quad \sqrt{2.667} = 1.633$$

$$\text{row } 2: \ \frac{(-1)^2 + (-1)^2 + 2^2}{3} = \frac{6}{3} = 2, \quad \sqrt{2} = 1.414$$

Divide each row by its own value (the std is also copied across columns):

$$\text{row 1} = \left[\tfrac{-2}{1.633}, \tfrac{0}{1.633}, \tfrac{2}{1.633}\right] = [-1.225,\ 0,\ 1.225]$$

$$\text{row 2} = \left[\tfrac{-1}{1.414}, \tfrac{-1}{1.414}, \tfrac{2}{1.414}\right] = [-0.707,\ -0.707,\ 1.414]$$

(Real implementations add a tiny number to the standard deviation to avoid dividing by zero.)

7.3 The silent bug: $(N, 1)$ minus (N)

Calculation

Predictions $\hat{y}$ have shape $(3, 1)$ and targets y have shape (3):

$$\hat{y} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}, \qquad y = [3,\ 4,\ 8].$$

Intended: compare each prediction with its own target (shapes made equal first, e.g. by flattening $\hat{y}$).

$$\text{differences} = (2-3,\ 4-4,\ 6-8) = (-1,\ 0,\ -2)$$

$$\text{squares} = (1,\ 0,\ 4)$$

$$\text{mean squared error} = \frac{1+0+4}{3} = 1.667$$

What the library does without being asked: $(3, 1)$ and (3) broadcast to $(3, 3)$. Every prediction is compared with every target:

$$\hat{y} - y = \begin{bmatrix} 2-3 & 2-4 & 2-8 \\ 4-3 & 4-4 & 4-8 \\ 6-3 & 6-4 & 6-8 \end{bmatrix} = \begin{bmatrix} -1 & -2 & -6 \\ 1 & 0 & -4 \\ 3 & 2 & -2 \end{bmatrix}.$$

$$\text{squares}: \ (1, 4, 36),\ (1, 0, 16),\ (9, 4, 4)$$

$$\text{sum} = 41 + 17 + 17 = 75$$

$$\text{mean} = \frac{75}{9} = 8.333$$

The loss is 8.333 instead of 1.667, with no error message. Always check shapes before computing a loss.

Where this appears in deep learning

Every bias addition, every layer normalization, and every attention mask relies on broadcasting. The $(N, 1)$ versus (N) mismatch is one of the most common silent bugs in training

code.

Key points to remember

- Compare shapes from the last dimension backwards. Equal or 1 is fine.
- A bias of shape (d) is repeated for every row of an (n, d) output.
- Use `keepdims=True` when taking a mean per row so the shape stays $(n, 1)$ and divides correctly.
- $(N, 1)$ with (N) gives (N, N), a plausible but wrong result.
- Always check shapes before reducing a loss.

Common mistake

- Believing a shape mismatch will always raise an error. Broadcasting often hides it.

8 Special matrices you will meet

8.1 Identity matrix

Calculation

I has 1 on the diagonal and 0 elsewhere, and leaves anything unchanged: $AI = IA = A$. For $x = [1, 2, -1]$ and I_3:

$$(xI)_1 = 1(1) + 2(0) + (-1)(0) = 1$$

$$(xI)_2 = 1(0) + 2(1) + (-1)(0) = 2$$

$$(xI)_3 = 1(0) + 2(0) + (-1)(1) = -1$$

So $xI = x$. A residual connection is the same idea: the input passes through unchanged and the layer's change is added on top.

8.2 Diagonal matrix: scaling each feature

Calculation

A diagonal matrix has numbers only on the diagonal. Multiplying on the right scales each *column*; this is how the learned scale in layer normalization works. Use $X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ and $D = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. Entry (i, j) of XD equals $x_{ij}d_j$ because all other terms are zero:

$$1(2) = 2 \qquad 2(0.5) = 1 \qquad 3(1) = 3$$

$$4(2) = 8 \qquad 5(0.5) = 2.5 \qquad 6(1) = 6$$

$$XD = \begin{bmatrix} 2 & 1 & 3 \\ 8 & 2.5 & 6 \end{bmatrix}.$$

(Multiplying on the left, DX, would scale *rows*.)

8.3 Lower-triangular matrix: the causal mask pattern

Calculation

A lower-triangular matrix has zeros above the diagonal. A causal mask for 3 tokens marks allowed positions with 1:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

Entry $(i, j) = 1$ exactly when $j \leq i$: token 1 sees only itself, token 2 sees tokens 1–2, token 3 sees all three.

8.4 Symmetric matrix: $X^T X$

Calculation

For $X = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 3 & 2 \end{bmatrix}$ we have $X^T = \begin{bmatrix} 1 & 0 \\ 2 & 3 \\ -1 & 2 \end{bmatrix}$, and $X^T X$ is $(3 \times 2)(2 \times 3) = 3 \times 3$. Entry (i, j) is column i of X dotted with column j of X:

$$(X^T X)_{11} = 1(1) + 0(0) = 1$$

$$(X^T X)_{12} = 1(2) + 0(3) = 2$$

$$(X^T X)_{13} = 1(-1) + 0(2) = -1$$

$$(X^T X)_{21} = 2(1) + 3(0) = 2$$

$$(X^T X)_{22} = 2(2) + 3(3) = 13$$

$$(X^T X)_{23} = 2(-1) + 3(2) = 4$$

$$(X^T X)_{31} = -1(1) + 2(0) = -1$$

$$(X^T X)_{32} = -1(2) + 2(3) = 4$$

$$(X^T X)_{33} = -1(-1) + 2(2) = 5$$

$$X^T X = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 13 & 4 \\ -1 & 4 & 5 \end{bmatrix}.$$

Entries $(1, 2)$ and $(2, 1)$ match, $(1, 3)$ and $(3, 1)$ match, $(2, 3)$ and $(3, 2)$ match. The matrix equals its own transpose.

Key points to remember

- Identity I: $AI = IA = A$. It is the matrix version of the number 1.
- Diagonal D: XD scales columns, DX scales rows.
- Lower-triangular: zeros above the diagonal; the causal-mask pattern.
- Symmetric: $A^T = A$. Any matrix times its own transpose is symmetric.

9 Putting it together

9.1 Forward pass of a two-layer network

Calculation

Three examples, three input features. Hidden layer with 2 neurons and ReLU, output layer with 1 neuron and sigmoid.

$$X = \begin{bmatrix} 2 & 6 & 0.6 \\ 5 & 7 & 0.9 \\ 1 & 5 & 0.5 \end{bmatrix},\ W_1 = \begin{bmatrix} 0.5 & -0.2 \\ 0.1 & 0.4 \\ 1.0 & 0.6 \end{bmatrix},\ b_1 = [-2,\ 0.2],\ W_2 = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix},\ b_2 = 0.3.$$

Shapes: XW_1 is $(3 \times 3)(3 \times 2) = 3 \times 2$; HW_2 is $(3 \times 2)(2 \times 1) = 3 \times 1$.

Step 1: XW_1**.**

$$(XW_1)_{11} = 2(0.5) + 6(0.1) + 0.6(1.0) = 1.0 + 0.6 + 0.6 = 2.2$$

$$(XW_1)_{12} = 2(-0.2) + 6(0.4) + 0.6(0.6) = -0.4 + 2.4 + 0.36 = 2.36$$

$$(XW_1)_{21} = 5(0.5) + 7(0.1) + 0.9(1.0) = 2.5 + 0.7 + 0.9 = 4.1$$

$$(XW_1)_{22} = 5(-0.2) + 7(0.4) + 0.9(0.6) = -1.0 + 2.8 + 0.54 = 2.34$$

$$(XW_1)_{31} = 1(0.5) + 5(0.1) + 0.5(1.0) = 0.5 + 0.5 + 0.5 = 1.5$$

$$(XW_1)_{32} = 1(-0.2) + 5(0.4) + 0.5(0.6) = -0.2 + 2.0 + 0.3 = 2.1$$

Step 2: add b_1 **to every row, then ReLU.**

$$\text{row 1}:\ [2.2 - 2,\ 2.36 + 0.2] = [0.2,\ 2.56]$$

$$\text{row 2}:\ [4.1 - 2,\ 2.34 + 0.2] = [2.1,\ 2.54]$$

$$\text{row 3}:\ [1.5 - 2,\ 2.1 + 0.2] = [-0.5,\ 2.3] \xrightarrow{\text{ReLU}} [0,\ 2.3]$$

$$H = \begin{bmatrix} 0.2 & 2.56 \\ 2.1 & 2.54 \\ 0 & 2.3 \end{bmatrix}.$$

Step 3: $HW_2 + b_2$**.**

$$z_1 = 0.2(1.0) + 2.56(0.5) + 0.3 = 0.2 + 1.28 + 0.3 = 1.78$$

$$z_2 = 2.1(1.0) + 2.54(0.5) + 0.3 = 2.1 + 1.27 + 0.3 = 3.67$$

$$z_3 = 0(1.0) + 2.3(0.5) + 0.3 = 0 + 1.15 + 0.3 = 1.45$$

Step 4: sigmoid $\sigma(z) = \dfrac{1}{1 + e^{-z}}$.

$$\sigma(1.78) = \frac{1}{1 + 0.1686} = 0.8557$$

$$\sigma(3.67) = \frac{1}{1 + 0.0255} = 0.9752$$

$$\sigma(1.45) = \frac{1}{1 + 0.2346} = 0.8100$$

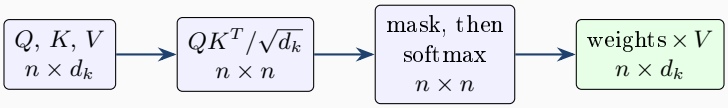
Final output: $(0.856,\ 0.975,\ 0.810)^T$, one number per example.

9.2 Masked self-attention for three tokens

Calculation

Single head, $d_k = 2$:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \; K = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}, \; V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 2 \end{bmatrix}.$$



Step 1: scores QK^T. $K^T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \end{bmatrix}$, so entry (i, j) is query i dotted with key j.

$$(1,1) = 1(1) + 0(0) = 1 \qquad (1,2) = 1(0) + 0(2) = 0 \qquad (1,3) = 1(1) + 0(1) = 1$$

$$(2,1) = 0(1) + 1(0) = 0 \qquad (2,2) = 0(0) + 1(2) = 2 \qquad (2,3) = 0(1) + 1(1) = 1$$

$$(3,1) = 1(1) + 1(0) = 1 \qquad (3,2) = 1(0) + 1(2) = 2 \qquad (3,3) = 1(1) + 1(1) = 2$$

Step 2: scale by $1/\sqrt{2} = 0.7071$.

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 2 & 2 \end{bmatrix} \times 0.7071 = \begin{bmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 1.4142 & 0.7071 \\ 0.7071 & 1.4142 & 1.4142 \end{bmatrix}.$$

Step 3: causal mask, then softmax per row. Future positions become $-\infty$.

row 1 : only token 1 $\Rightarrow$ [1, 0, 0]

row 2 : (0, 1.4142) : $e^0 = 1$, $e^{1.4142} = 4.1133$, sum $= 5.1133$

weights $= (0.1956, 0.8044)$

row 3 : (0.7071, 1.4142, 1.4142) : $e^{0.7071} = 2.0281$, 4.1133, 4.1133, sum $= 10.2547$

weights $= (0.1978, 0.4011, 0.4011)$

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0.1956 & 0.8044 & 0 \\ 0.1978 & 0.4011 & 0.4011 \end{bmatrix}.$$

Step 4: output AV. Each output row mixes the rows of V using that token's weights.

row $1 = 1(1, 0) = (1, 0)$

row $2 = 0.1956(1, 0) + 0.8044(0, 1) = (0.1956, 0.8044)$

row $3 = 0.1978(1, 0) + 0.4011(0, 1) + 0.4011(2, 2)$

$= (0.1978 + 0.8022, \; 0.4011 + 0.8022) = (1.0000, 1.2033)$

$$AV = \begin{bmatrix} 1 & 0 \\ 0.1956 & 0.8044 \\ 1.0000 & 1.2033 \end{bmatrix}.$$

Token 1 sees only itself, so its output is exactly v_1.

Key points to remember

- Before debugging any layer, write the shape of every input, weight, bias, and output.
- Network layer: matrix product, add bias (broadcast), apply activation (elementwise).
- Attention: QK^T, scale, mask with $-\infty$, softmax, then multiply by V.
- In attention the shapes go $(n, d_k) \to (n, n) \to (n, d_k)$. The (n, n) matrix is why long inputs are expensive.
- Compare computed results with tolerances, never exact equality.

10 Formula sheet

$$(A + B)_{ij} = a_{ij} + b_{ij}$$

$$(\alpha A)_{ij} = \alpha a_{ij}$$

$$(AB)_{ij} = \sum_k a_{ik} b_{kj}$$

$$(AB)^T = B^T A^T$$

$$(AB)C = A(BC)$$

$$A(B + C) = AB + AC$$

$$(A \odot B)_{ij} = a_{ij} b_{ij}$$

$$\text{work of } (m \times k)(k \times n) = mkn \text{ multiplications}$$

$$\text{layer parameters} = \text{inputs} \times \text{outputs} + \text{outputs}$$

$$Z = XW: \quad dW = X^T dZ, \quad dX = dZ\, W^T$$

$$\text{attention} = \text{softmax}\Big(\frac{QK^T}{\sqrt{d_k}} + \text{mask}\Big)V$$

11 Practice problems

Show the shape at each step.

1. A network takes 8 examples with 6 features, goes through layers $6 \to 4 \to 2$. Give each tensor's shape and count the parameters.

2. Compute $\mathrm{ReLU}(XW + b)$ for $X = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$, $W = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}$, $b = [0,\ 1,\ -1]$.

3. For $X = \begin{bmatrix} 1 & 2 \\ 3 & 0 \end{bmatrix}$ and $dZ = \begin{bmatrix} 1 & -1 \\ 0 & 2 \end{bmatrix}$ compute $dW = X^T dZ$.

4. A gate $g = [0.5,\ 1,\ 0]$ multiplies $v = [4,\ -2,\ 7]$ elementwise. Find the result.

5. Token 2 can see tokens 1 and 2, with scores $(2,\ 0)$. Find its attention weights.

6. Normalize the token $[3,\ 3,\ 9]$ with layer normalization (subtract mean, divide by standard deviation).

7. Count the multiplications in QK^T for $n = 6$ and for $n = 12$, with $d_k = 4$.

8. Explain why a causal mask must use $-\infty$ before softmax and not a $0/1$ multiplication.

12 Answers

1. X: 8×6. After layer 1: $(8 \times 6)(6 \times 4) = 8 \times 4$. After layer 2: $(8 \times 4)(4 \times 2) = 8 \times 2$.

$$\text{layer } 1 : 6 \cdot 4 + 4 = 28$$

$$\text{layer } 2 : 4 \cdot 2 + 2 = 10$$

$$\text{total} = 38$$

2. XW:

$$\text{row } 1 :\ [1(1) + (-1)(0),\ 1(2) + (-1)(1),\ 1(0) + (-1)(1)] = [1,\ 1,\ -1]$$

$$\text{row } 2 :\ [2(1) + 0(0),\ 2(2) + 0(1),\ 2(0) + 0(1)] = [2,\ 4,\ 0]$$

Add b: row 1 $= [1,\ 2,\ -2]$, row 2 $= [2,\ 5,\ -1]$. ReLU: $\begin{bmatrix} 1 & 2 & 0 \\ 2 & 5 & 0 \end{bmatrix}$.

3. $X^T = \begin{bmatrix} 1 & 3 \\ 2 & 0 \end{bmatrix}$.

$$dW_{11} = 1(1) + 3(0) = 1 \qquad dW_{12} = 1(-1) + 3(2) = 5$$

$$dW_{21} = 2(1) + 0(0) = 2 \qquad dW_{22} = 2(-1) + 0(2) = -2$$

$dW = \begin{bmatrix} 1 & 5 \\ 2 & -2 \end{bmatrix}$.

4. $[0.5(4),\ 1(-2),\ 0(7)] = [2,\ -2,\ 0]$.

5. $e^2 = 7.389$, $e^0 = 1$, sum $= 8.389$. Weights $= (0.881,\ 0.119)$.

6. Mean $= \frac{3+3+9}{3} = 5$. Centered: $[-2,\ -2,\ 4]$. Variance $= \frac{4+4+16}{3} = 8$, standard deviation $= 2.828$. Result: $[-0.707,\ -0.707,\ 1.414]$.

7. $n = 6$: $6 \cdot 4 \cdot 6 = 144$. $n = 12$: $12 \cdot 4 \cdot 12 = 576$. Doubling n gives 4 times the work.

8. A 0/1 multiplication turns forbidden scores into 0, and $e^0 = 1$, so those positions still receive weight. Setting them to $-\infty$ gives $e^{-\infty} = 0$, so they receive none.

13 Self-check

- I can tell from shapes alone whether a product is allowed and what shape it gives.
- I can compute a matrix product entry by entry, and also as a mixture of rows.
- I can explain why $AB \neq BA$ and why stacked linear layers collapse.
- I can compute $dW = X^T dZ$ and $dX = dZ\, W^T$ by hand.
- I can tell `A * B` from `A @ B` and use each correctly.
- I can build a causal mask correctly, with $-\infty$ before softmax.
- I can predict the shape that broadcasting will produce.
- I can trace a full forward pass of a small network and one attention head by hand.